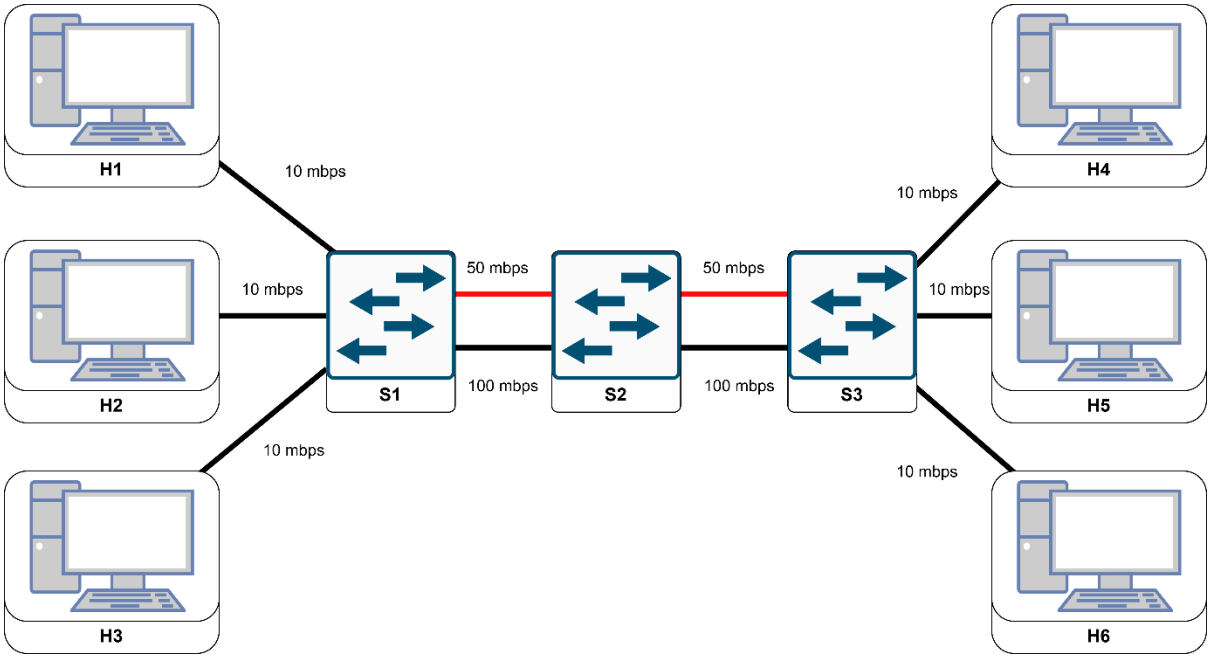
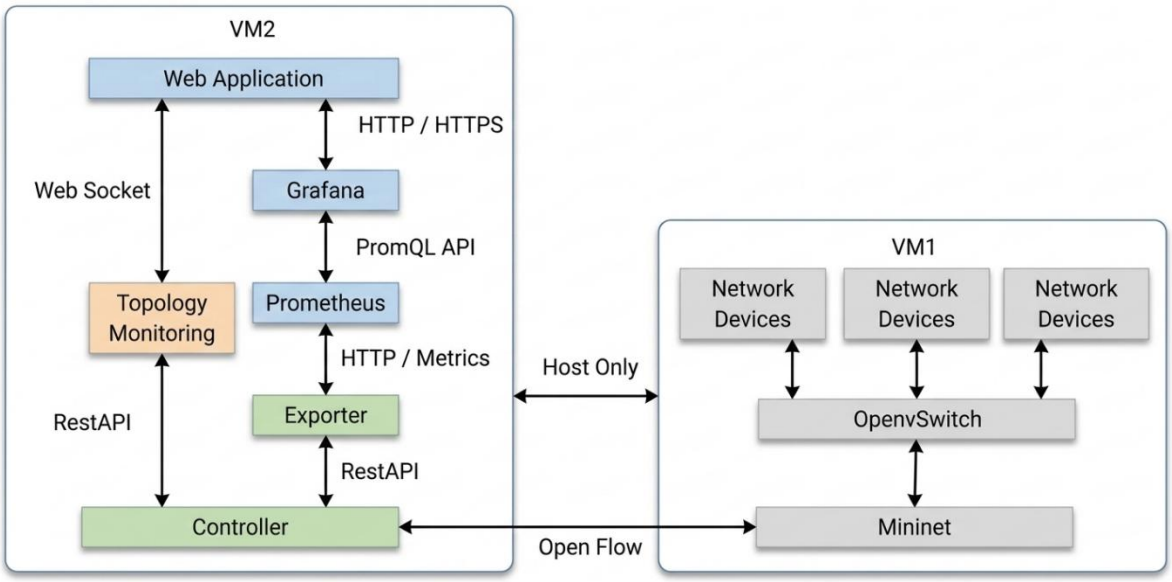


1. Requirements
   - VM 1
     a) Ubuntu Server 20.04/22.04
     b) Ryu Controller
     c) Grafana
     d) Prometheus
     e) Web Service
   - VM 2
     a) Ubuntu Server 20.04/22.04
     b) Open vSwitch (OVS)
     c) Mininet
2. Setup Mininet dan Ryu dengan ketentuan
   - Peserta harus membuat dan menjalankan Ryu Controller sebagai container dalam docker.
   - Pastikan Ryu Controller dapat menerima koneksi dari Mininet.
   - Verifikasi konektivitas dengan menggunakan perintah pingall dalam Mininet.
   - Gunakan topologi linear dengan 3 Switch: S1 (Edge-A), S2 (Core), dan S3 (Edge-B)
   - Konfigurasikan dua subnet yang berbeda :
     a) **Subnet A (10.0.1.0/24)**: Terdiri dari H1, H2, dan H5 yang terhubung ke S1.
     b) **Subnet B (10.0.2.0/24):** Terdiri dari H3, H4, dan H6 yang terhubung ke S3.

- Link antar Switch menggunakan bandwidth 100 Mbps, sedangkan link Host-ke-Switch menggunakan 10 Mbps
- Implementasikan logika controller Ryu yang menggabungkan **4** konsep
  a) **Layer 3 Switch**: Flow entry harus dipasang berdasarkan IP Source dan IP Destination untuk protokol ICMP.
  b) **Layer 2 Switch**: Untuk trafik TCP dan UDP, flow entry harus dipasang secara spesifik mencakup nomor port (Source Port dan Destination Port).
  c) **Flow Timeout**: Setiap flow entry yang dipasang secara otomatis oleh controller harus memiliki batas waktu Idle Timeout = 30 detik dan Hard Timeout = 120 detik.
  d) **Backup Link** : Link antar switch harus tedapat backup yang bandwidth-nya setengah dari Main Link dan akan aktif ketika Main Link down,

3. Pembuktian
   - **Verifikasi Layer 3 Switch**: Lakukan **`ping`** dari H1 ke H3 (lintas subnet), kemudian tunjukkan pada **`ovs-ofctl`** dump-flows bahwa switch menyimpan flow dengan kriteria nw_src dan nw_dst.
   - **Verifikasi Layer 2 Switch :**
     a) Jalankan **iperf** TCP antara H1 dan H3, lalu tunjukkan bahwa flow entry mencakup detail tp_dst=5001.
     b) Jalankan **iperf** UDP antara H5 dan H6, lalu tunjukkan bahwa flow entry menggunakan kriteria udp dan port yang sesuai.
   - **Verifikasi Flow Timeout :** Tunjukkan jumlah flow entry pada switch saat ada trafik aktif. Kemudian biarkan jaringan idle selama >30 detik dan tunjukkan menggunakan ovs-ofctl bahwa flow tersebut telah terhapus otomatis (hanya tersisa table-miss flow).
   - **Backup Link** : Buat salah satu link antar switch down, lalu tunjukkan pada **`ovs-ofctl`** dump-flows bahwa trafik berpindah dari Main Link ke Backup Link.

4. Monitoring task

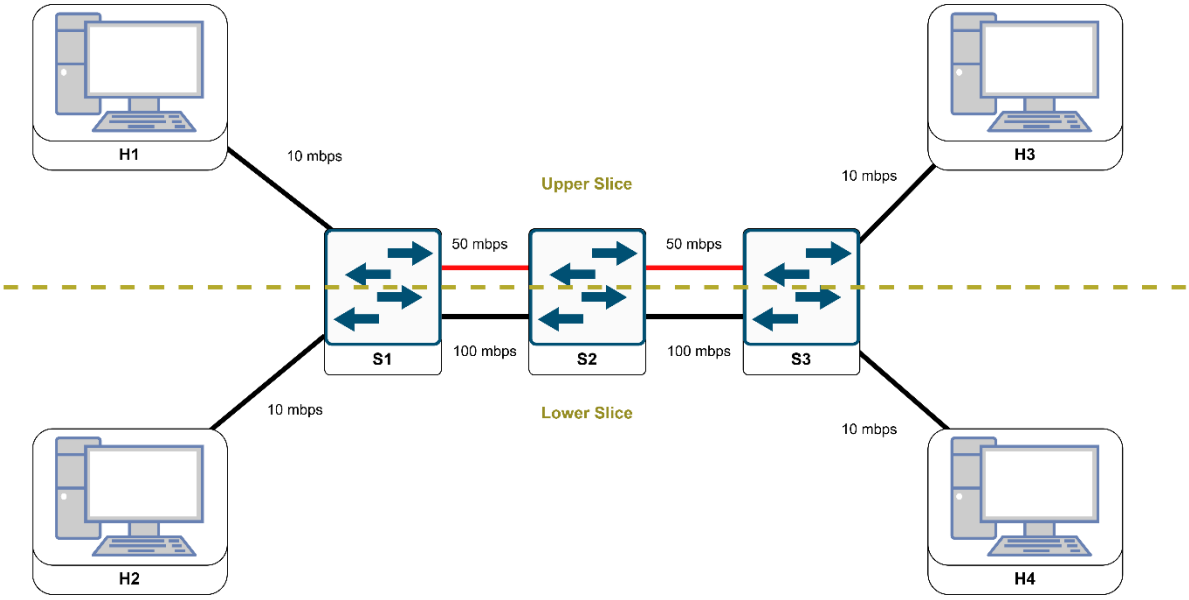


Buatlah dashboard berbasis web yang terhubung ke VM 2 untuk :

- Memonitoring jaringan, dapat memvisualisasikan switch host secara real time
- Menampilkan informasi bandwidth dan statistik link menggunakan data dari Exporter dan Grafana
- Advance configuration pada setiap node, bisa menampilkan terminal containernet, bisa x-term setiap host, dan bisa Up/down enable/disable node is a plus point.

5. Bonus Challenge

Buatkan topologi & controller yang terpisah dari soal sebelumnya yang mengimplementasikan **Network Slicing** dengan ketentuan berikut :



- Lower Slice : Menggunakan **Backup Link**. Digunakan untuk trafik UDP port 9999.
- Upper Slice : Menggunakan **Backup Link** untuk semua trafik lainnya
- Gunakan **OVS Queues** agar Upper Slice tidak mengambil bandwidth dari Lower Slice

Untuk memastikan network slicing berhasil, lakukan pengujian berikut :

- Jalankan **iperf UDP** pada port 9999 dari H2 ke H4 (Lower Slice)
- Secara bersamaan, jalankan **iperf TCP** biasa dari H1 ke H3 (Upper Slice).
- Cek dashboard **Grafana**. Seharusnya terlihat dua grafik berbeda, satu stabil di angka 100 Mbps dan satu lagi di 50 Mbps.
- Cek flow table `ovs-ofctl` dump-flows lalu pastikan terdapat *flow entry* yang spesifik mengarahkan trafik UDP port 9999 ke **queue 0** dan trafik lainnya ke **queue 1**.

6. Dokumentasi & Evaluasi
    - Setiap langkah harus terdokumentasi dengan baik dalam laporan.
    - Pastikan setiap konfigurasi dapat direplikasi oleh orang lain.
    - Buat repositori di github untuk dokumentasinya lengkap & terstruktur dengan README penjelasan arsitektur, backed API, step-step deployment & testing soal
    - Pastikan setiap anggota berkontribusi dan menjadi kolabolator di repositori tersebut